

说明

CH32L103 系列是基于 RISC-V 内核的工业级低功耗通用微控制器，集成了多种低功耗模式（Sleep、Stop、Standby）。在实际应用中，系统功耗不仅取决于芯片本身，同时取决于软件对电源模式、时钟系统和外设的合理配置。本文旨在帮助了解 CH32L103 的功耗来源，低功耗模式的选择逻辑及低功耗实现策略。文档涵盖低功耗模式对比、电流来源分析、休眠模式选择指南及运行阶段优化方法。

适用范围

适用范围	系列
通用 MCU	CH32L103 等

目录

说明	1
目录	2
表格索引	2
图片索引	2
第 1 章 低功耗模式与电流	1
1.1 低功耗模式	1
1.2 电流来源	2
1.2.1 运行模式下	2
1.2.2 低功耗模式期间	2
1.3 休眠模式选择	2
1.3.1 根据工作状态	2
1.3.2 完整周期比较	3
1.3.3 应用选择示例	3
1.4 低功耗模式的定时唤醒-LPTIM	4
第 2 章 运行阶段优化	5
2.1 主频与 HSI 低功耗	5
2.1.1 按任务类型调整主频	5
2.1.2 HSI-LP 与 HSE-LP	5
2.1.3 PLL 与外设时钟	5
2.2 FLASH 与 SRAM	5
2.3 减少 CPU 参与时间	6
历史版本	7
声明	8

表格索引

低功耗模式一览	1
电流主要消耗	2

图片索引

三种模式周期任务电流	3
LPTIM 低功耗唤醒配置	4
不同主频运行电流与持续时间	5
FLASH 进入低功耗	6

第1章 低功耗模式与电流

1.1 低功耗模式

不同模式在功耗、唤醒时间、时钟配置和状态保持方面存在显著差异。Stop 模式进一步细分为 Stop1~4，提供不同级别的功耗与 RAM 保持策略。选择模式时，需在功耗、唤醒速度、状态保持三者之间进行权衡。

表 1-1 低功耗模式一览

模式与关键配置	状态	可工作	唤醒源	唤醒后
Sleep SLEEPDEEP=0 执行 WFI 或 WFE	内核停止运行。其他时钟不因进入 Sleep 自动关闭，SRAM、寄存器和 I/O 状态保持。	已使能并获得时钟的外设可继续工作。进入前关闭不用的外设时钟，以减少相应电流。	WFI：由中断唤醒。 WFE：由唤醒事件唤醒；也可按手册配置由中断相关事件唤醒。	不复位。WFI 唤醒后先执行中断服务函数；WFE 事件唤醒后继续执行，具体路径取决于事件配置。
Stop1 SLEEPDEEP=1, PDDS=0 LPDS=0; LDO 节能关闭 执行 WFI 或 WFE	HSE、HSI、PLL 和外设时钟关闭；SRAM、寄存器和 I/O 状态保持。Stop1~4 的时钟停止方式相同。	可运行模块：IWDG、RTC、LSI/LSE、LPTIM。	外部中断或事件、PVD 输出、RTC 闹钟事件、USB 唤醒信号、USB PD 唤醒信号、CMP、LPTIM；NRST 也可退出 Stop，但属于复位路径。	非复位唤醒后从原程序继续，HSI 为默认系统时钟；任务需要 HSE、PLL 或特定外设时钟时，由程序重新恢复。NRST 路径执行复位。
Stop2 SLEEPDEEP=1, PDDS=0 LPDS=0; AUTO_LDO_EC=1 或 LDO_EC=1 执行 WFI 或 WFE				
Stop3 SLEEPDEEP=1, PDDS=0 LPDS=1, RAMLV=0 进入前 FLASH_LP=10b 执行 WFI 或 WFE				
Stop4 SLEEPDEEP=1, PDDS=0 LPDS=1, RAMLV=1 进入前 FLASH_LP=10b 执行 WFI 或 WFE				
Standby SLEEPDEEP=1, PDDS=1 执行 WFI 或 WFE 进入前 FLASH_LP=10b	HSE、HSI、PLL 和外设时钟关闭，主电压调节器关闭；除唤醒电路与后备域外，其余电路断电。可按配置保持 2 KB、18 KB 或两部分 SRAM；普通寄存器和运行上下文不保留。	IWDG、RTC、LSI/LSE。	WKUP 引脚上升沿、RTC 闹钟事件、NRST、IWDG 复位；EXTI0~EXTI17 的外部事件也可唤醒，这里是外部事件路径，不是普通 EXTI 中断。	所有退出路径均按电源复位流程启动，不从进入 Standby 的位置继续执行。

1.2 电流来源

CH32L103 的电流消耗可分为动态电流（由内核、总线和数字外设运行引起）和静态电流（由模拟模块、内部调节器和漏电流引起）。在运行模式和低功耗模式下，电流来源及其在总功耗中的占比差异较大。

表 1-2 电流主要消耗

主要来源	产生电流主要活动	影响电流的配置/条件
内核与总线	执行指令、读写寄存器、搬运数据	系统主频、HSILP、PLLON
时钟系统	HSI/HSE、PLL 振荡和时钟分配	时钟源、频率、PLL 与模块时钟使能
FLASH 与 SRAM	Flash 取指和读常量；SRAM 读写与数据保持	代码执行位置、Flash 低功耗、RAM 保持范围
数字与模拟外设	外设等模块工作所需电流	外设时钟使能、工作时间、工作模式
I/O 接口	输入电路、内部上下拉和输出驱动工作	引脚模式、上下拉与输出翻转
内部供电调节等	调节器供电及 SRAM 状态保持	LDO_EC、LPDS、RAMLV 与保留区域

1.2.1 运行模式下

运行时，内核的电流消耗主要由取指和数据访问构成。与此同时，时钟信号通过时钟树分发到总线和各已使能的外设，驱动定时器、USART、SPI、I2C 等模块工作。程序若反复查询同一状态位而不做有效工作，CPU、Flash 和总线仍保持活动，这部分轮询开销不产生实际结果，但电流照常消耗。ADC、CMP 等模拟模块在使能时也有独立的电流消耗。

因此，同样的主频下，代码从 Flash 还是 RAM 执行、模块时钟开启、是否反复轮询、I/O 是否持续翻转，都会改变运行电流。降低主频可能降低瞬时电流，但也可能延长内核和外设工作的时间；关闭不使用的模块时钟与缩短任务时间解决的是不同来源的功耗。

1.2.2 低功耗模式期间

Sleep 减少的主要是内核执行相关的活动。Stop 保留 SRAM、寄存器、低速计时与唤醒所需电路；不同 Stop 档位还改变内部调节器和 RAM 的供电状态。Standby RTC、唤醒电路以及选中的 SRAM 保持区域仍可能耗电。

1.3 休眠模式选择

1.3.1 根据工作状态

首先考虑休眠期间需要保留哪些功能，唤醒后从哪里继续执行。

Sleep: 内核停止，但外设时钟按原配置继续工作。如果休眠期间仍需高速时钟驱动的外设（如 USART 接收、ADC 连续转换、SPI 从机等），Sleep 通常是唯一可用的低功耗模式。

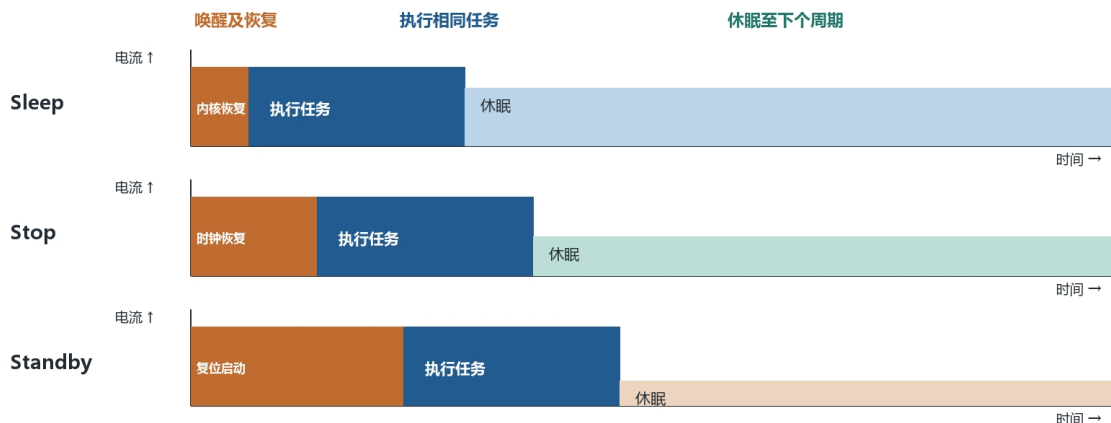
Stop: 关闭高速时钟（HSE/HSI/PLL 域），SRAM、寄存器和 I/O 状态保持。如果休眠期间只需低速计时（RTC、LPTIM）或事件唤醒（EXTI、CMP、PVD 等），且唤醒后需要从原程序继续执行，Stop 可以考虑。

Standby：主电压调节器关闭，大部分电路断电，可选保持部分 SRAM。唤醒后按电源复位流程启动。只有当唤醒源可用、启动时间可接受，并且程序状态能重建或通过允许的保留方式取得时，Standby 可以考虑。

1.3.2 完整周期比较

对周期性任务，以三种模式使用同一任务、同一唤醒周期比较为例，每一周期都包含唤醒与恢复、执行任务、再次休眠三段。Sleep 模式休眠电流较高，但内核恢复快；Stop 的休眠电流介于 sleep 和 standby 中间，唤醒后需恢复任务所需时钟和外设；Standby 模式可以进一步降低功耗，却要完成复位启动和状态重建。

图 1-1 三种模式周期任务电流



图中，横向是时间，纵向是电流。同一周期平均电流取决于三段电流与各自持续时间共同结果，不单纯比较休眠电流或比较唤醒电流峰值。

周期较短时：恢复与工作占比高，Stop 或 Standby 模式节省休眠电流的时间有限；此时 Sleep 可能得到更低的周期平均电流。

周期拉长：恢复成本分摊到更多休眠时间，Stop、Standby 的低休眠电流更容易形成净收益。

任务几乎不停执行：完整周期中没有足够长的休眠段。此时持续运行并降低运行电流也可进入选择，例如 HSI 低功耗 1 MHz，或 RAM 执行同时使 Flash 进入低功耗。它们改变的是持续工作的电流；与 Sleep、Stop、Standby 比较时，仍以相同任务和相同时间范围内的平均电流为准。

1.3.3 应用选择示例

◆ 场景：智能门锁在无人操作时处于待机状态，指纹模块需周期性唤醒检测是否有手指按压，检测间隔约 200ms~500ms 检测一次手指按压。每次检测时间短，检测完成后立即回到休眠。

Stop4 关闭 HSE/HSI/PLL，仅保留低速模块，SRAM 和寄存器内容不丢失，唤醒后从原程序继续，HSI 作为默认系统时钟；LPTIM 可用 LSE 或 LSI 作为计数时钟，二者在 Stop 模式下仍工作，因此能周期唤醒 CPU。Sleep 不关闭高速时钟，周期内时钟系统持续耗电；Standby 唤醒后按电源复位流程启动，指纹模块需重新初始化。门锁待机时间占绝大部分，检测时间短，Stop4 把待机功耗压到最低，LPTIM 把周期定时交给硬件，CPU 只在检测时短暂工作。Stop4 配置为 SLEEPDEEP=1、PDDS=0、LPDS=1 电压调节器工作在低功耗模式，RAMLV=1 RAM 在低电压模式，预置 FLASH_LP[1:0]=10b。

◆ 场景：电池供电的电子墨水屏显示设备，内容更新间隔远大于屏幕刷新时间。刷新完成后，屏幕在断电条件下保持显示内容。设备在两次更新之间没有持续性的计算或通信任务，MCU 进入待机模式，等待下一次定时或外部事件唤醒。

MCU 进入 CH32L103 的 Standby 模式。RTC 配置为周期闹钟，作为主要唤醒源。跨复位周期需要保留的数据存放在保持 RAM 中，通过配置 R2KSTY=1 保持 2KB RAM 不掉电。程序从保持 RAM 读取数据，

通过 SPI 驱动墨水屏完成刷新，然后再次配置 RTC 闹钟和 RAM 保持，进入 Standby。若存在外部事件唤醒需求，可将对应引脚配置为 EXTI 或 WKUP 唤醒源。

1.4 低功耗模式的定时唤醒-LPTIM

LPTIM (Low-Power Timer) 是专为超低功耗场景设计的定时器模块。LPTIM 的核心价值在于能够在除待机模式外的所有电源模式下运行，以极低的功耗维持计时功能。

LPTIM 具有内部-LSE (32.768kHz)、LSI (约 40kHz)、HSI 或 HB 时钟和外部 LPTIM 输入上的外部时钟等多种时钟源。

低功耗模式对 LPTIM 的影响：

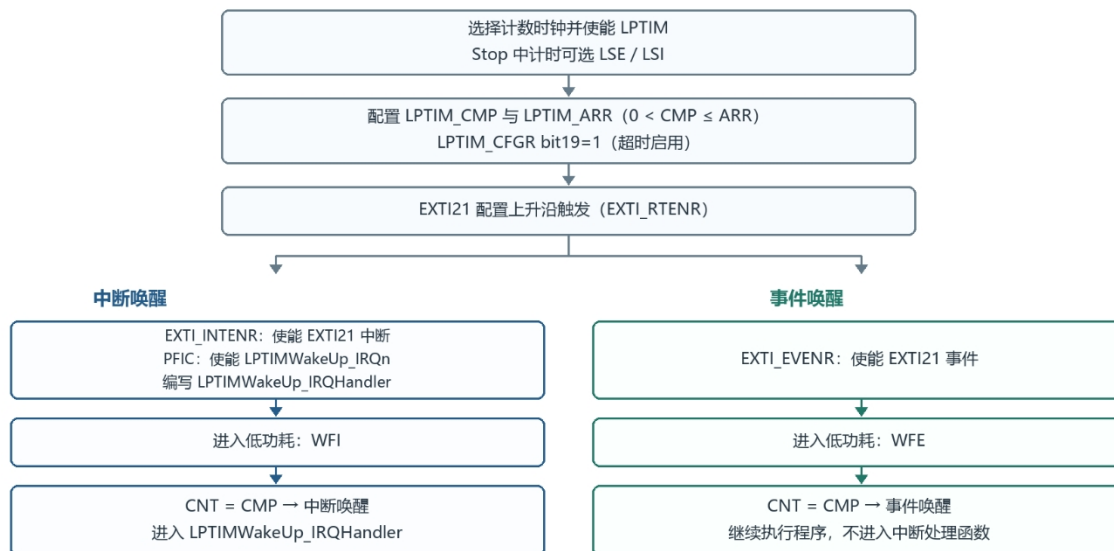
SLEEP 模式：无影响，LPTIM 中断会导致设备退出睡眠模式。

STOP 模式：LPTIM 外围设备在由 LSE 或 LSI 计时时处于活动状态，LPTIM 中断导致设备退出停止模式。

STANDBY 模式：LPTIM 外围设备已断电，必须在退出待机模式后重新初始化。

CH32L103 LPTIM 低功耗唤醒配置流程如下：

图 1-2 LPTIM 低功耗唤醒配置



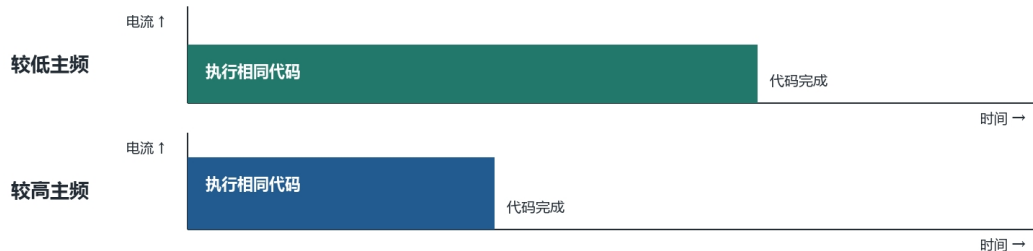
第2章 运行阶段优化

2.1 主频与 HSI 低功耗

2.1.1 按任务类型调整主频

主频提高后，内核与总线每秒运行的时钟周期增多，取指、读写数据等活动更频繁，因而同一配置下的运行电流通常会上升。主频降低时，这部分活动减少，运行电流可能下降；但每个时钟周期更长，同一段代码一般需要更久才能执行完。

图 2-1 不同主频运行电流与持续时间



运行电流下降的幅度未必与主频下降的幅度相同：在代码执行期间，HSI、Flash、已开启的外设以及内部供电电路仍消耗电流。因此，低主频虽然减小部分运行电流，却让这些电路保持工作更久；降低电流得到的收益，可能被延长的执行时间抵消。反过来，较高主频虽抬高运行电流，只要同一段代码足够快地完成，其执行过程的总耗电也可能更低。

2.1.2 HSI-LP 与 HSE-LP

HSI-LP：CH32L103 的 HSI 内部低功耗模式，开启（RCC_CTLR 的 HSILP 位置 1）后 HSI 输出由 8 MHz 降为 1 MHz。程序仍可执行，适合需要持续运行、且 1 MHz 已满足执行时间与外设时序的阶段，HSILP 可降低这一阶段的运行电流；若程序本来很快就结束，收益取决于低速运行持续的时间。进入 HSILP 时，应将芯片对应的低功耗校准值装入 HSITRIM，并核对串口波特率、定时与软件超时。

HSE-LP：晶体振荡器可以通过 RCC_CTRL 寄存器将 HSELP 位置 1 进入低功耗模式。若外部晶体、时钟精度或启动时间对工作有要求，需在目标配置与工作条件下验证；HSE 未使用时，开启 HSELP 不能代替关闭不用的 HSE。

2.1.3 PLL 与外设时钟

关闭不需要的 PLL。若当前工作和其他功能已不需要 PLL 输出，可在完成时钟切换后关闭 PLL，减少低速运行期间的额外电流。

外设总线按需要分频。CH32L103 可分别配置 HCLK、PCLK1、PCLK2。内核需较高频率时，可把 PB 总线降到满足接口时序的频率；完全不用的外设则关闭对应模块时钟。

减少重复启动和切换。短任务之间反复启动 PLL、等待稳定或重新初始化外设，可能消耗掉低频运行带来的收益。

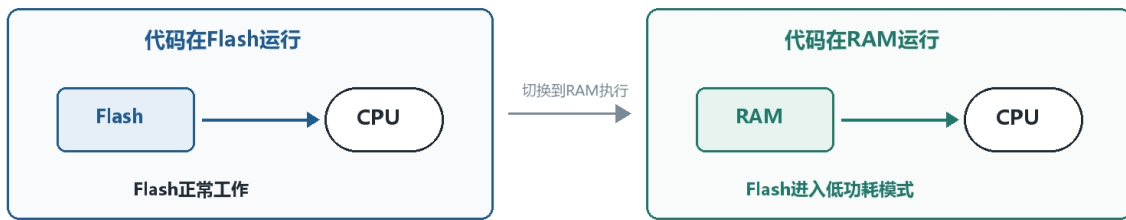
RTC 的时钟源可在 LSE、LSI 或 HSE/128 之间选择。若 RTC 使用 HSE 分频作为时钟源，则 RTC 的持续计时依赖 HSE，不能仅为降低运行电流而关闭 HSE。长期计时可在初始化时选择满足精度要求的 LSE 或 LSI，避免单为计时保留高速时钟。

2.2 FLASH 与 SRAM

CH32L103 有 FLASH 低功耗模式配置位，可以将程序从 Flash 搬运到 RAM 中执行，同时通过 FLASH_LP 配置使 Flash 进入低功耗模式。这样，CPU 持续运行期间不再从 Flash 取指，Flash 的读取

活动和静态电流一并降低，运行电流随之下降。该方案结合 2.1.2 节的 HSILP，可进一步降低运行电流。适用于运行时间较长、规模较小、且期间不需要访问 Flash 的程序段。

图 2-2 FLASH 进入低功耗



2.3 减少 CPU 参与时间

在运行阶段，ADC 规则组连续采样、USART 收发和 SPI 通信等数据量较大的任务，可以使用 DMA 完成外设与 RAM 之间的数据搬运。CPU 负责配置 DMA 通道、传输方向、外设与存储器地址、数据宽度、传输数量及中断条件；传输开始后，DMA 根据外设请求完成数据搬运。CPU 可在传输过半、传输完成或发生错误时集中处理数据，无需逐个数据读取或写入外设寄存器。

CPU 逐项搬运数据时，需要反复读取状态、访问外设寄存器、读写 RAM 并更新缓冲区位置，同时伴随指令执行、Flash 取指和总线访问。改用 DMA 后，这部分 CPU 执行和 Flash 访问可以减少，连续数据传输中的中断次数也可以降低。不过，DMA 控制器、外设和总线时钟在传输期间仍然工作，DMA 与 CPU 访问 SRAM 时也需要仲裁。因此，DMA 更适合连续采样、较长数据包或需要循环缓冲的任务；对于数据量很小、传输不频繁的任务，其配置和结束处理开销可能抵消部分收益。

历史版本

日期	版本	变更内容
2026/9/21	V1.0	初版发行

声明

本手册仅供参考。作者在不另行通知的情况下，可随时进行修改。